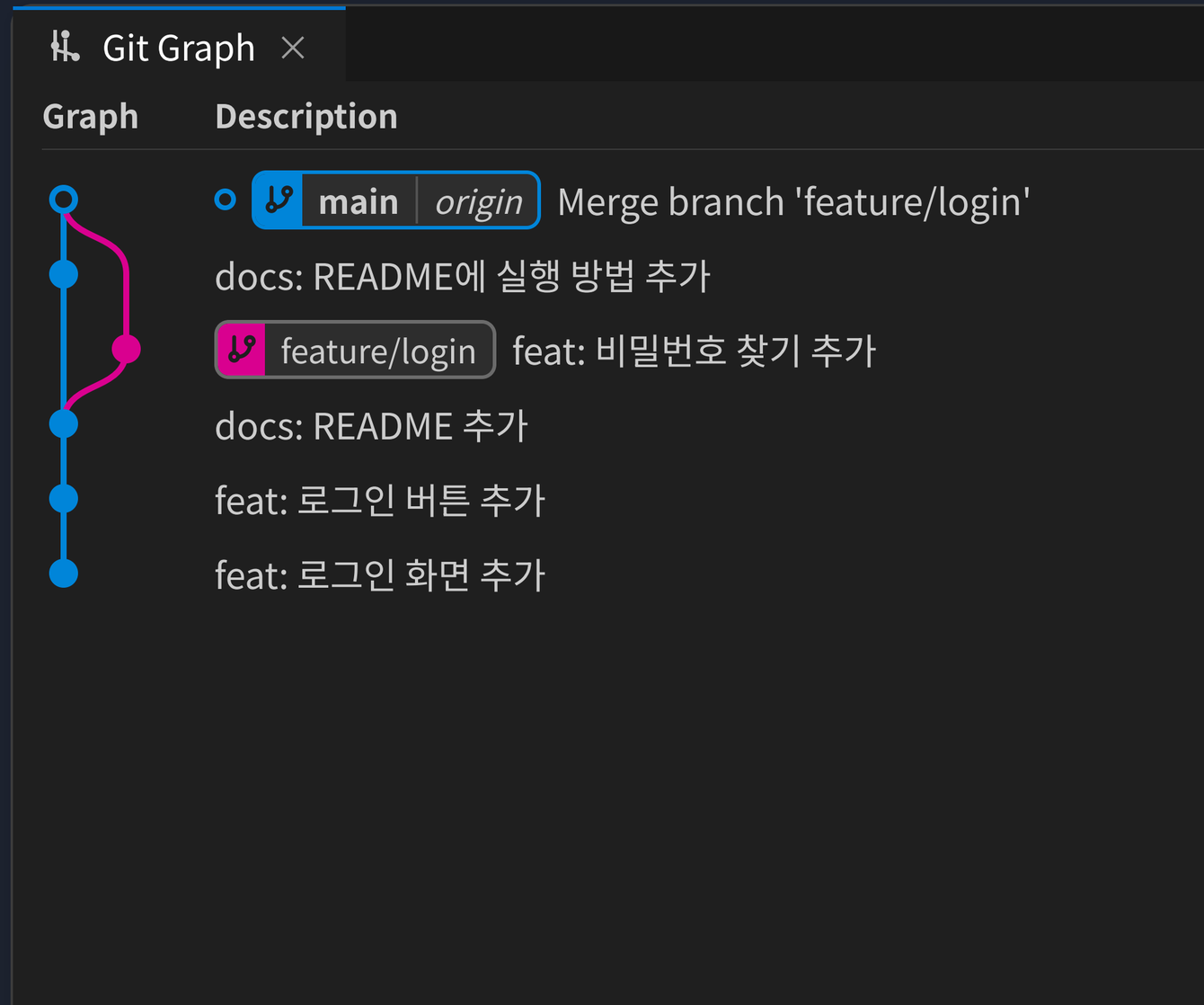
KANT AX/AI 엔지니어 트랙 · 05

Git Graph로 기록 보기

터미널에서 친 결과를
VS Code에서 그림으로 확인해요



05 · Git Graph

오늘의 순서

들어가기	터미널 기록과 Git Graph가 어떻게 다른지	3–4쪽
1. 설치하고 열기	확장 탭에서 설치하고, 저장소 폴더에서 열기	5–6쪽
2. 화면 읽기	칸, 점, 굵은 이름표, origin/main	7–10쪽
3. 명령 치고 확인하기	commit부터 merge까지 열 번, 커밋 상세 보기	11–21쪽
4. 막힐 때와 실제 상황	메뉴 대신 터미널, 안 보일 때, 그림 보고 명령 고르기	22–26쪽

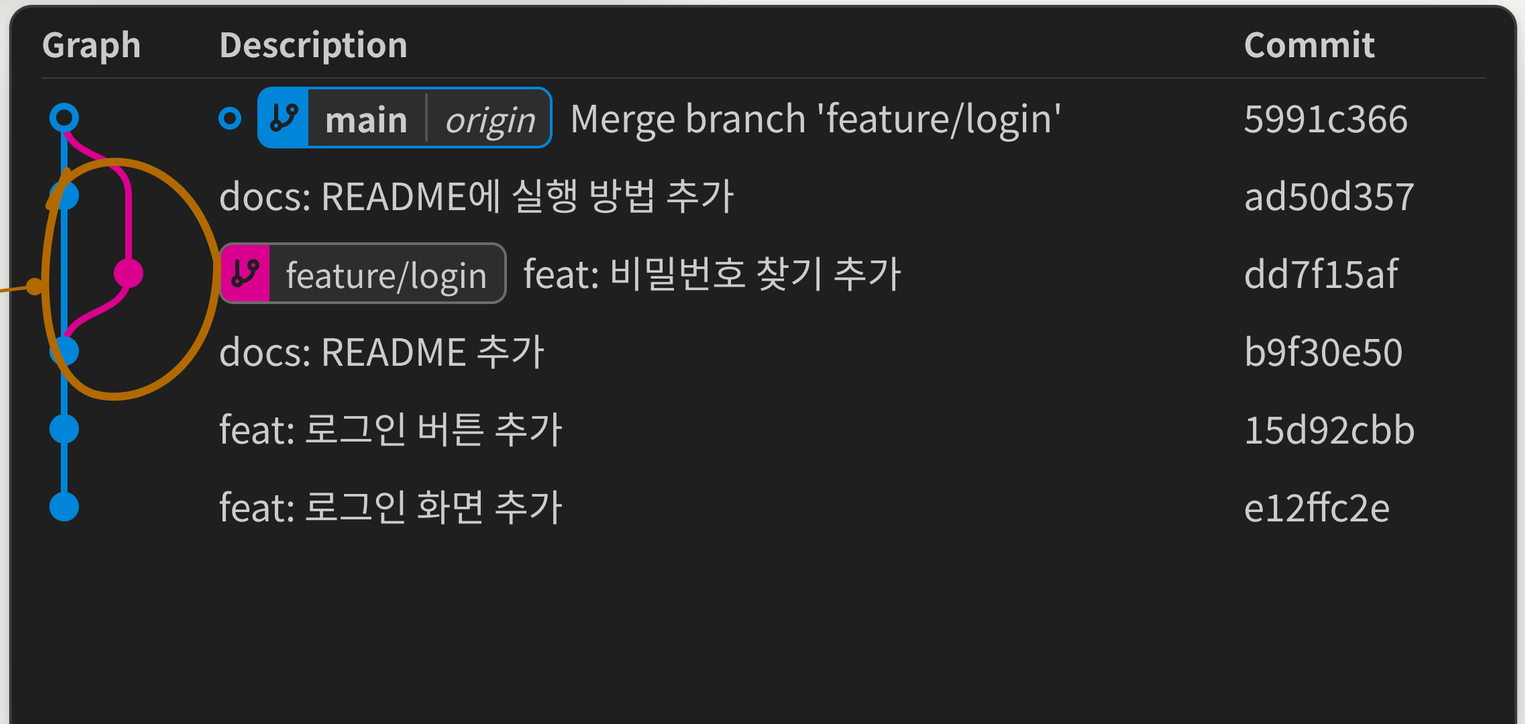
왜 필요할까요

git log --oneline으로는 갈라진 길이 안 보여요

터미널

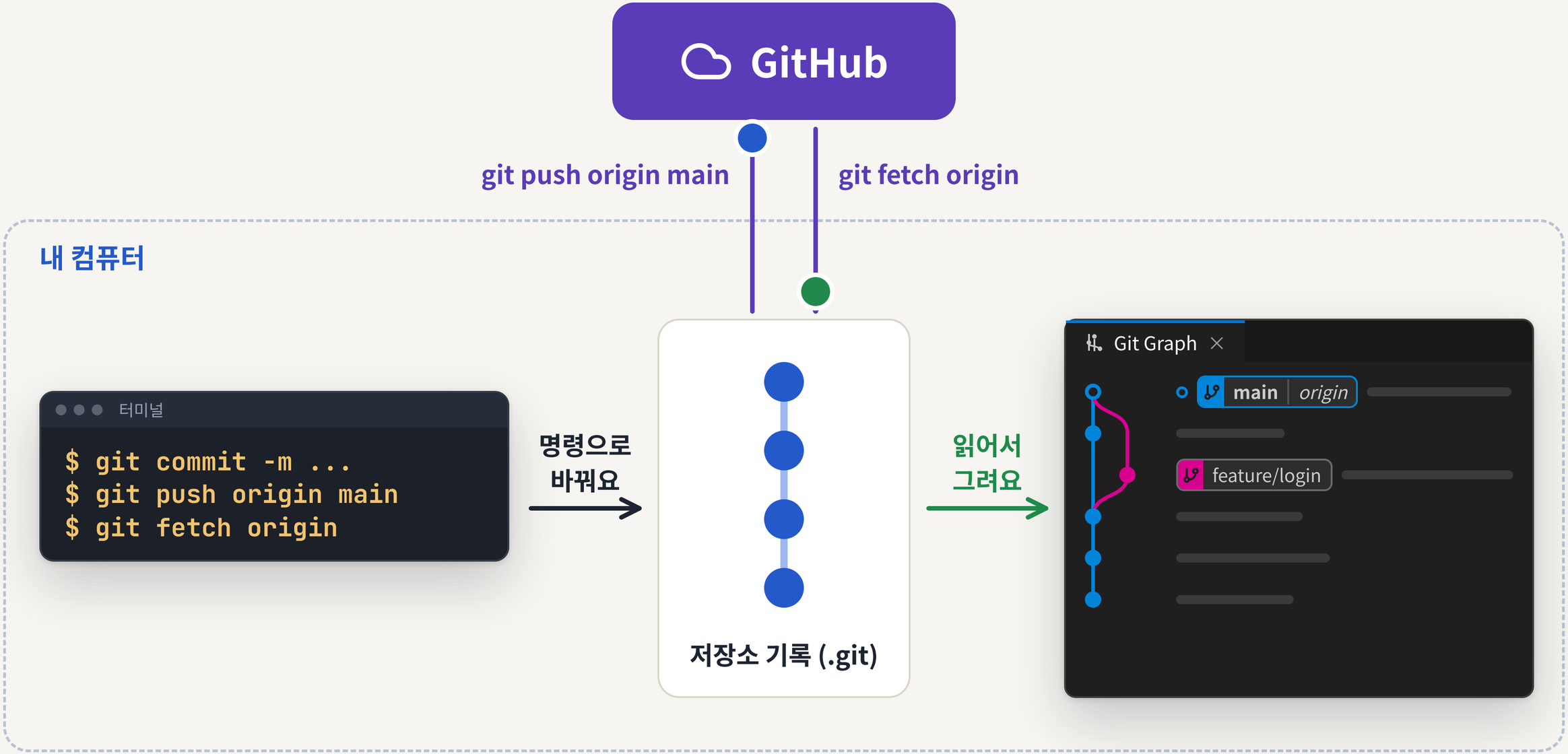
```
$ git log --oneline
5991c36 (HEAD -> main, origin/main) Merge branch 'feature/login'
ad50d35 docs: README에 실행 방법 추가
dd7f15a (feature/login) feat: 비밀번호 찾기 추가
b9f30e5 docs: README 추가
15d92cb feat: 로그인 버튼 추가
e12ffc2 feat: 로그인 화면 추가
```

feature/login에서 갈라졌다가 merge로 다시 만나요



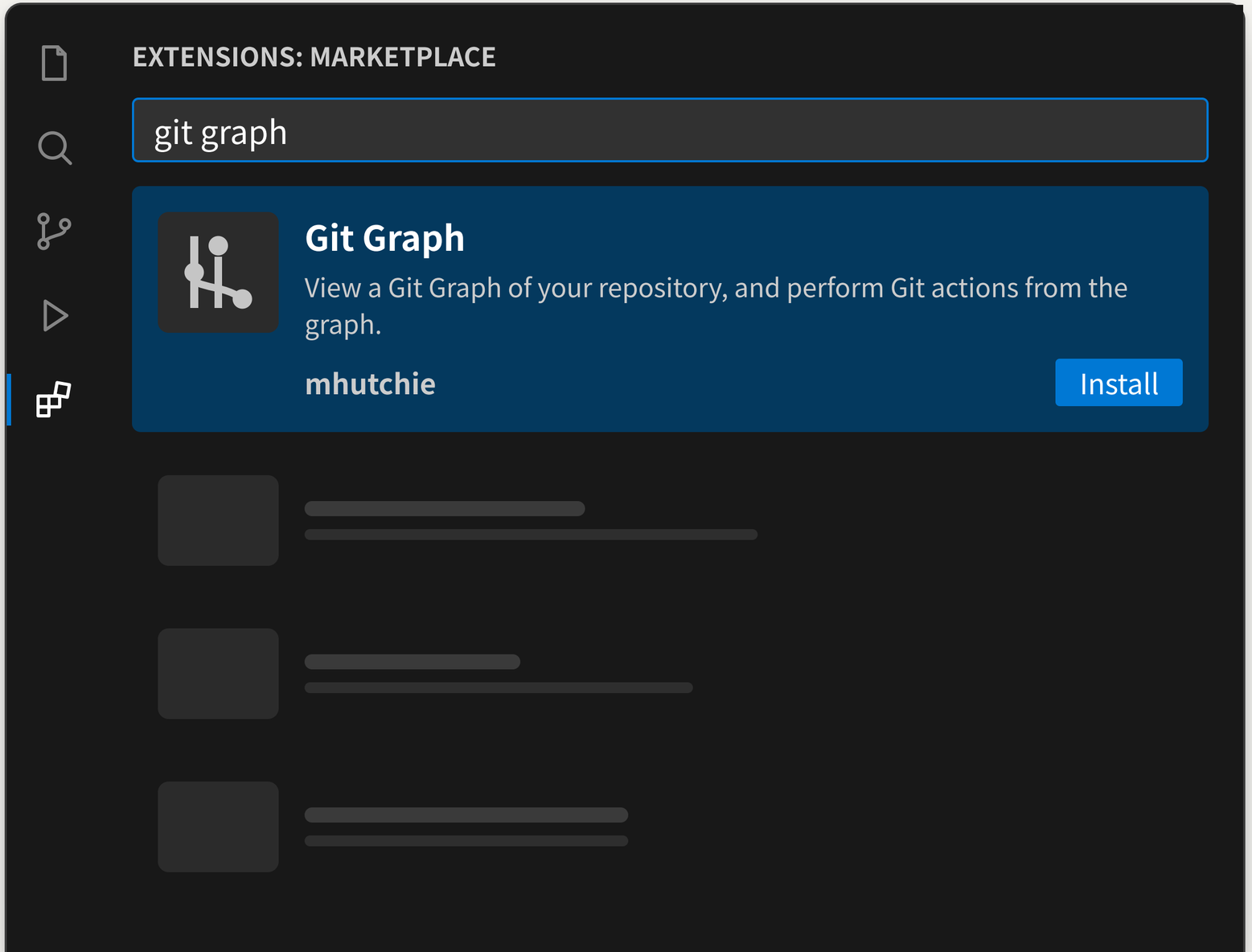
큰 그림

Git Graph는 내 컴퓨터에 있는 기록을 읽어서 그려요



설치하고 열기

확장 탭에서 Git Graph를 찾아 설치해요

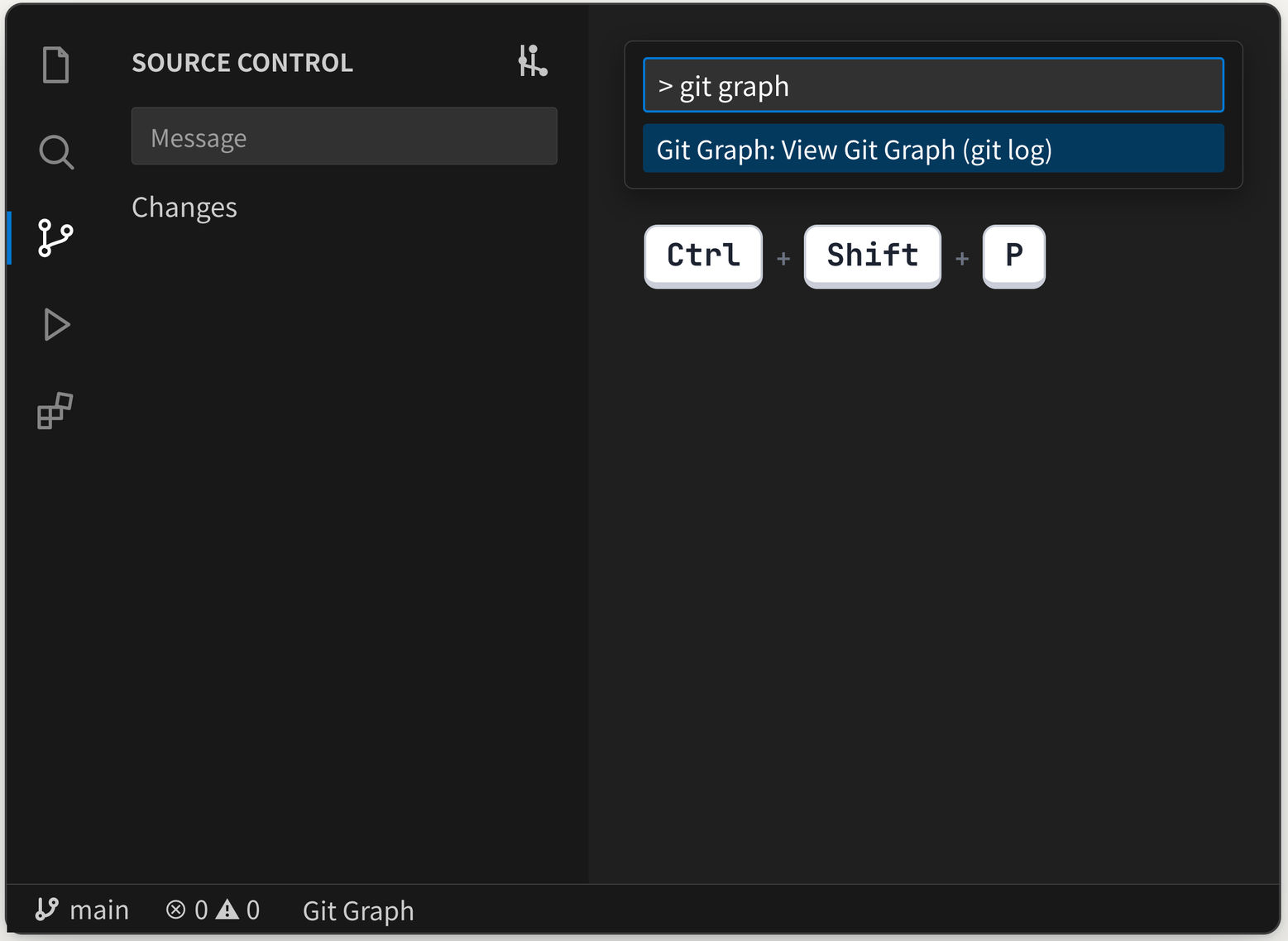


1. **왼쪽 막대의 확장 아이콘**
   Windows Ctrl+Shift+X
   Mac Cmd+Shift+X
2. **검색창에 git graph**
3. **만든 사람이 mhutchie인지 보고 Install**

설치가 끝나면 Install 자리에
톱니바퀴가 나와요.

설치하고 열기

저장소 폴더를 열고, 아래 막대의 Git Graph를 눌러요



1. **아래 상태 표시줄의 Git Graph**
   가장 빨라요

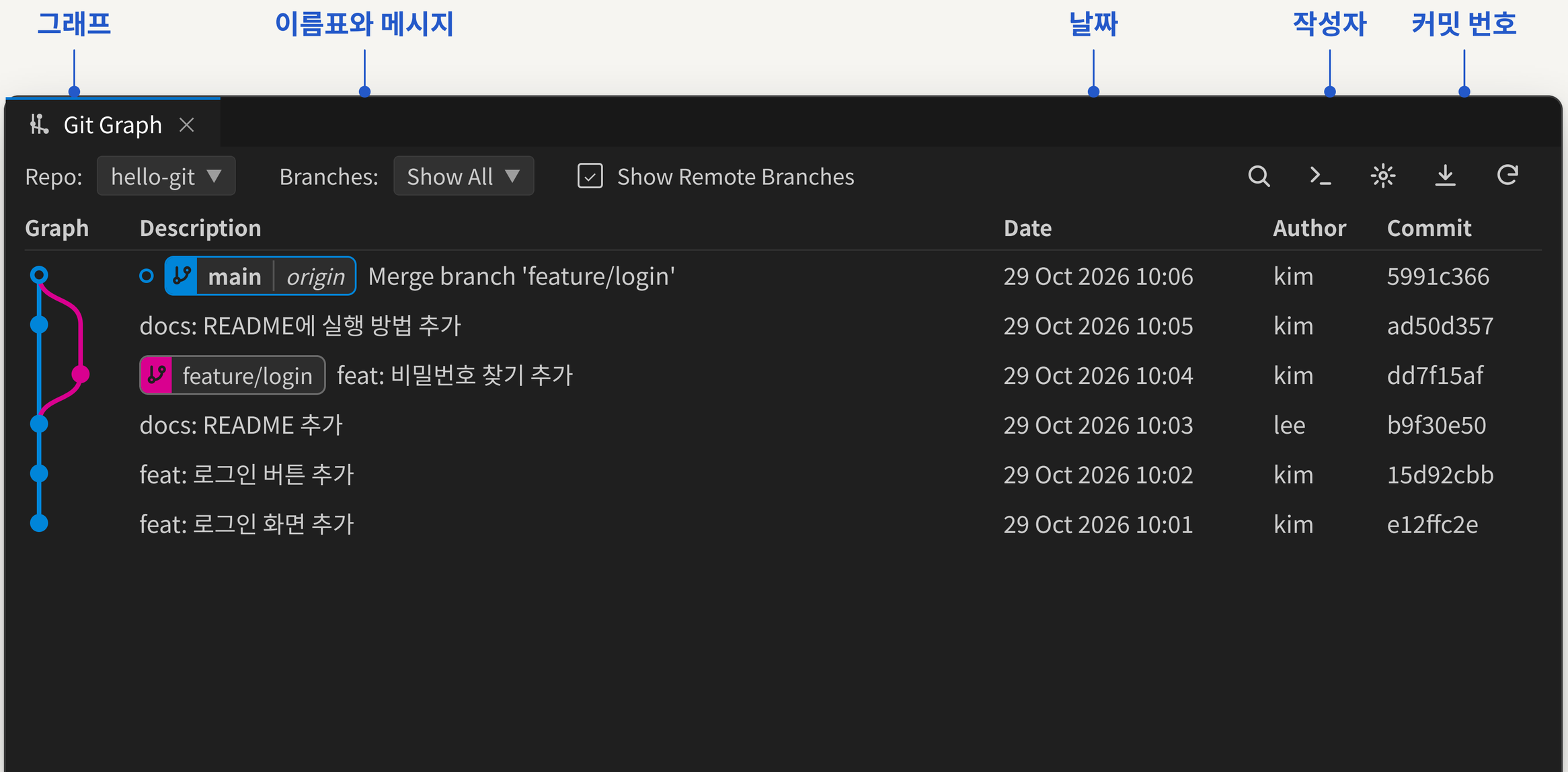
2. **소스 제어 탭 위의 그래프 아이콘**

3. **명령 팔레트에서 git graph**
   Windows Ctrl+Shift+P
   Mac Cmd+Shift+P

먼저 File > Open Folder로 hello-git 폴더를 열어 둬요.

화면 읽기

한 줄이 커밋 하나, 맨 위가 가장 최근이에요



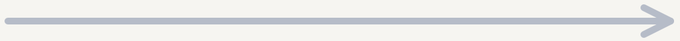
화면 읽기

03의 그림을 세우면 Git Graph가 돼요

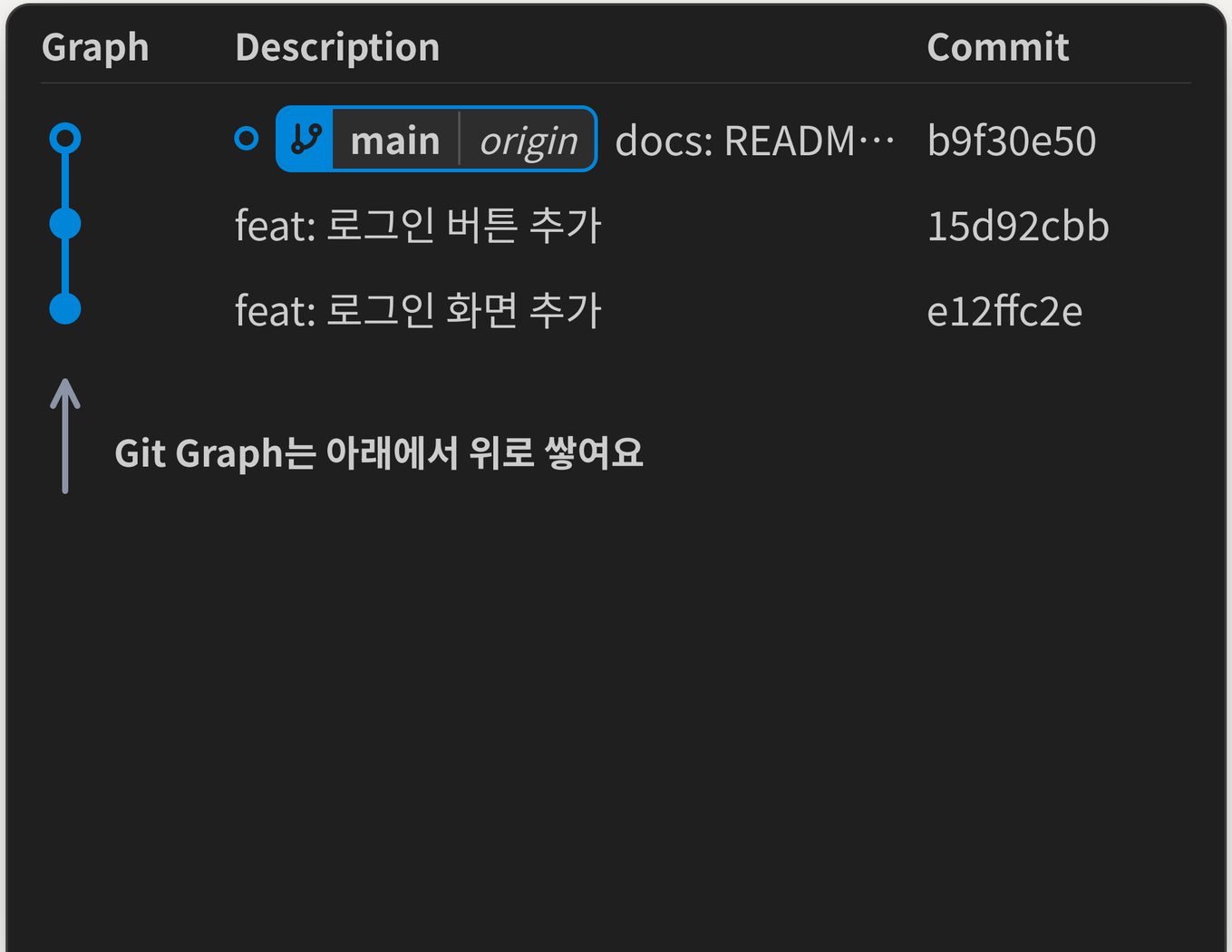
03 슬라이드



왼쪽에서 오른쪽으로 쌓여요



커밋 번호는 8자리로 나와요.
터미널의 7자리와 앞부분이 같아요.



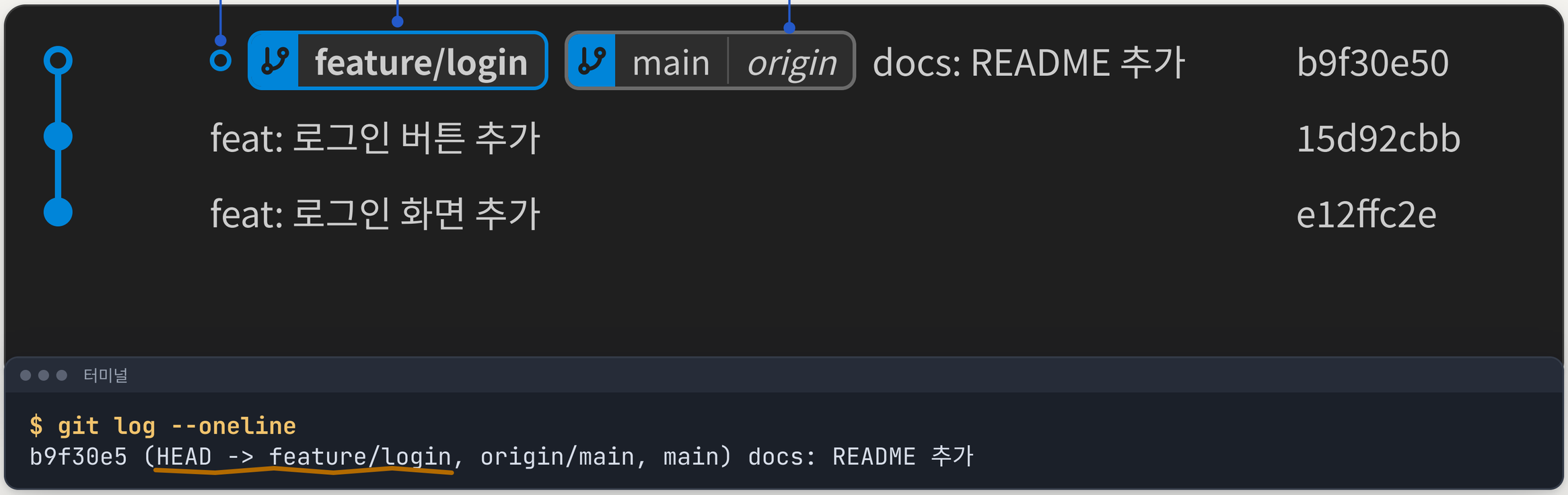
화면 읽기

굵은 이름표가 지금 있는 브랜치예요

속이 빈 점
HEAD가 가리키는 커밋

굵은 글씨, 색 테두리
지금 있는 브랜치, HEAD ->

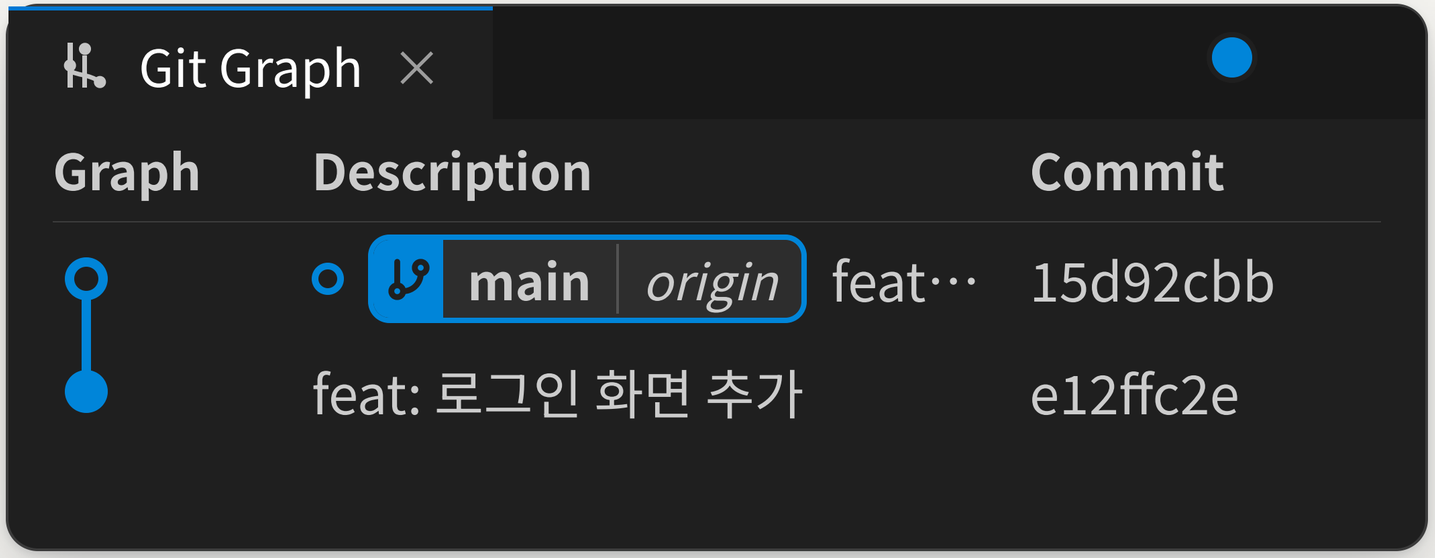
기울어진 origin
origin/main도 같은 커밋



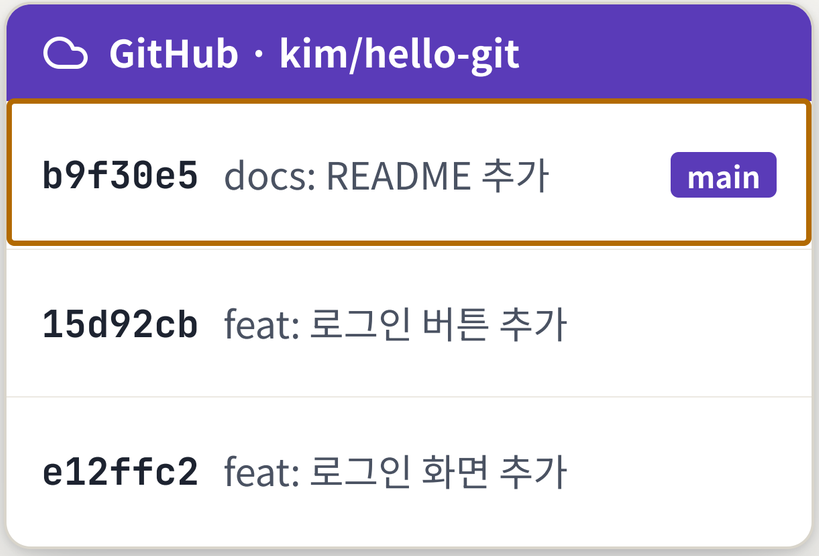
화면 읽기

origin/main은 마지막으로 GitHub과 주고받은 위치예요

내 Git Graph



GitHub의 실제 main

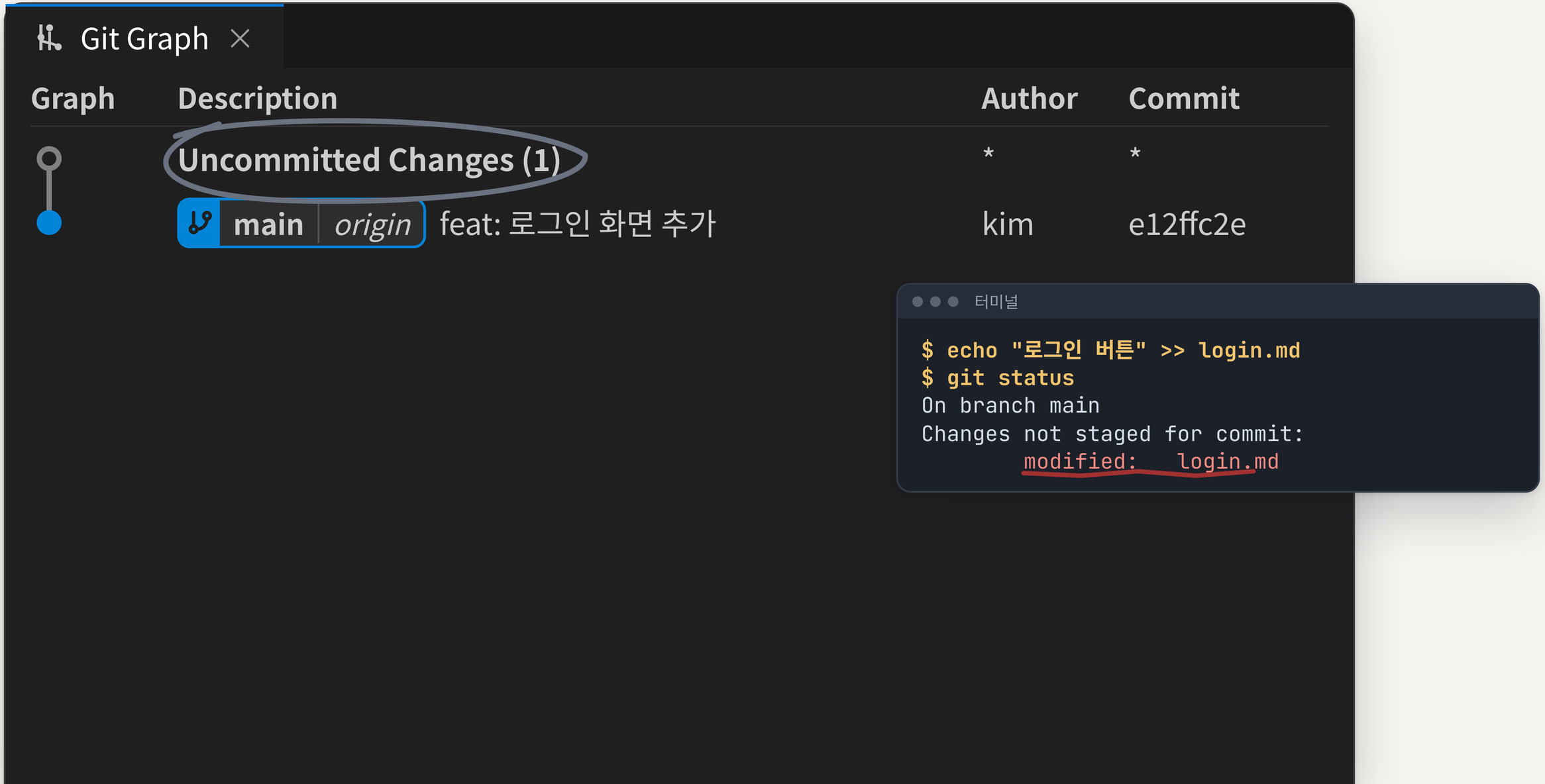


팀원 lee가 올린 b9f30e5는 GitHub에만 있고 내 Git Graph에는 아직 없어요.

`git fetch origin`을 쳐야 내 컴퓨터로 받아지고 그림에 나타나요.

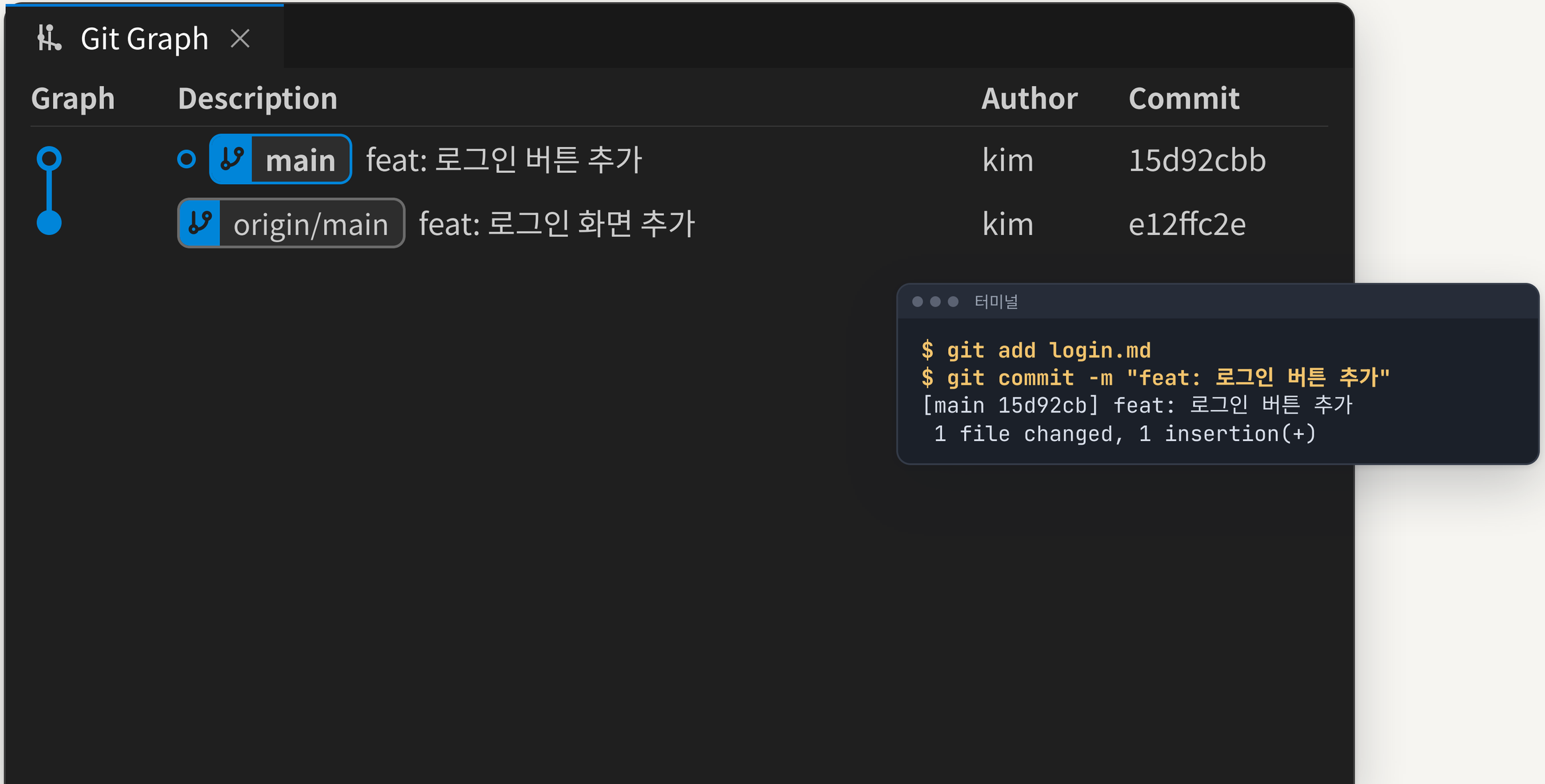
실습 1 · 파일을 고치면

맨 위에 회색 빈 점이 생겨요



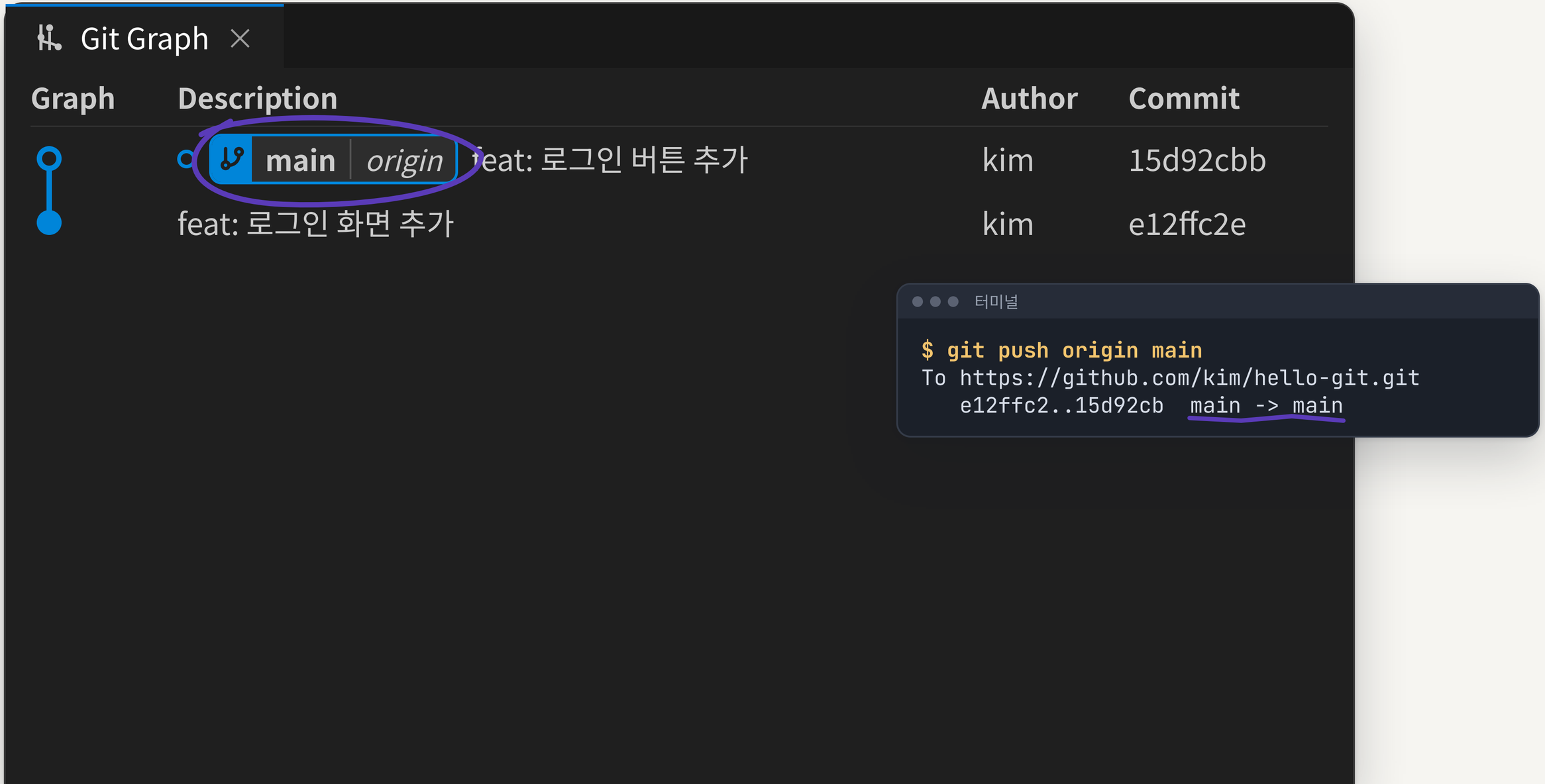
실습 2 · git commit

새 점이 위에 생기고 main이 따라 올라가요



실습 3 · git push origin main

origin이 main 옆에 붙어요



실습 4 · git fetch origin

팀원 커밋이 받아지고 origin/main이 main보다 위로 가요

Git Graph

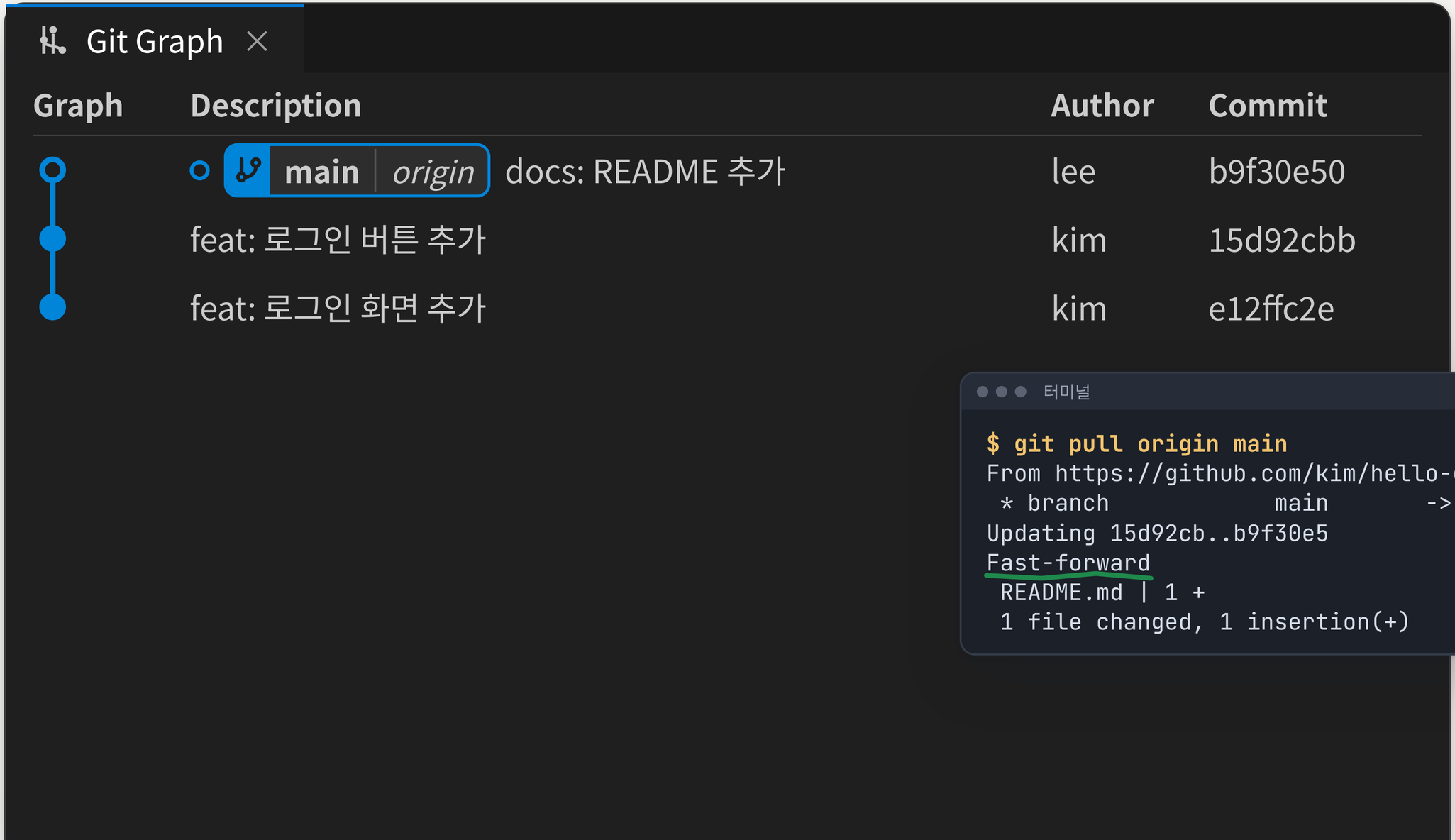
Graph	Description	Author	Commit
	origin/main docs: README 추가	lee	b9f30e50
	main feat: 로그인 버튼 추가	kim	15d92cbb
	feat: 로그인 화면 추가	kim	e12ffc2e

터미널

```
# 팀원 lee가 GitHub에 push했어요
$ git fetch origin
From https://github.com/kim/hello-git
   15d92cb..b9f30e5  main       -> origin/main
```

실습 5 · git pull origin main

main이 올라가서 origin과 다시 붙어요

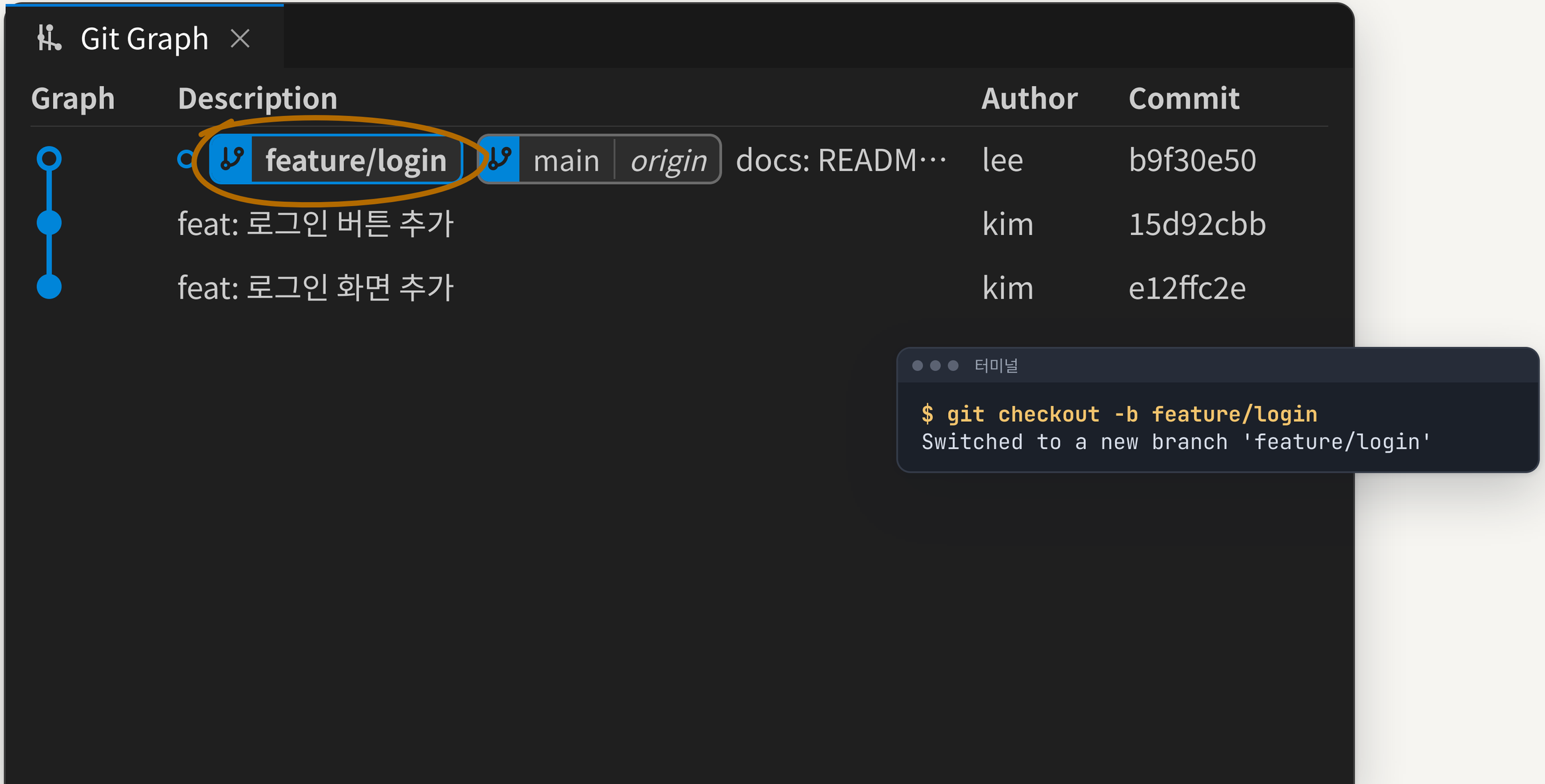


터미널

```
$ git pull origin main
From https://github.com/kim/hello-git
 * branch            main       -> FETCH_HEAD
Updating 15d92cb..b9f30e5
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
```

실습 6 · git checkout -b feature/login

같은 점에 이름표가 하나 더 붙어요



실습 7 · 브랜치에서 git commit

feature/login만 위로 올라가요

Git Graph

Graph	Description	Author	Commit
●	feature/login feat: 비밀번호 찾기 추가	kim	dd7f15af
●	main origin docs: README 추가	lee	b9f30e50
●	feat: 로그인 버튼 추가	kim	15d92cbb
●	feat: 로그인 화면 추가	kim	e12ffc2e

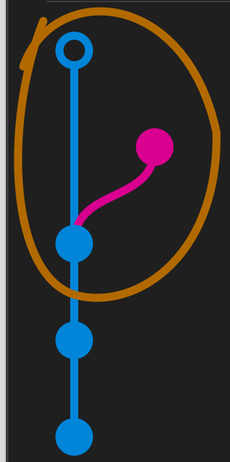
터미널

```
$ echo "비밀번호 찾기" >> login.md
$ git add login.md
$ git commit -m "feat: 비밀번호 찾기 추가"
[feature/login dd7f15a] feat: 비밀번호 찾기 추가
 1 file changed, 1 insertion(+)
```

실습 8 · main으로 돌아가 git commit

길이 둘로 갈라져요

Git Graph



Graph	Description	Author	Commit
	main docs: README에 실행 방법 추가	kim	ad50d357
	feature/login feat: 비밀번호 찾기 추가	kim	dd7f15af
	origin/main docs: README 추가	lee	b9f30e50
	feat: 로그인 버튼 추가	kim	15d92cbb
	feat: 로그인 화면 추가	kim	e12ffc2e

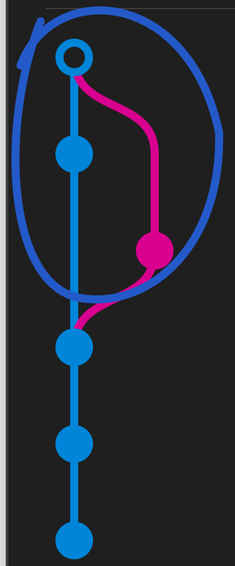
터미널

```
$ git checkout main
Switched to branch 'main'
$ echo "실행 방법은 준비 중이에요" >> README.md
$ git add README.md
$ git commit -m "docs: README에 실행 방법 추가"
[main ad50d35] docs: README에 실행 방법 추가
```

실습 9 · git merge feature/login

두 선이 merge 커밋에서 만나요

Git Graph

Graph	Description	Author	Commit
	main Merge branch 'feature/login'	kim	5991c366
	docs: README에 실행 방법 추가	kim	ad50d357
	feature/login feat: 비밀번호 찾기 추가	kim	dd7f15af
	origin/main docs: README 추가	lee	b9f30e50
	feat: 로그인 버튼 추가	kim	15d92cbb
	feat: 로그인 화면 추가	kim	e12ffc2e

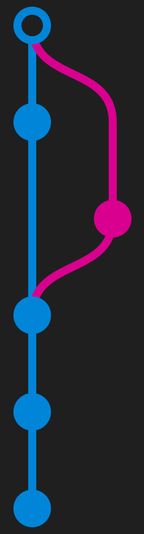
터미널

```
$ git merge feature/login
# vim이 열리면 Esc, :wq, Enter
Merge made by the 'ort' strategy.
 login.md | 1 +
 1 file changed, 1 insertion(+)
```

실습 10 · 다시 git push origin main

main | origin이 맨 위에 같이 있어요

Git Graph

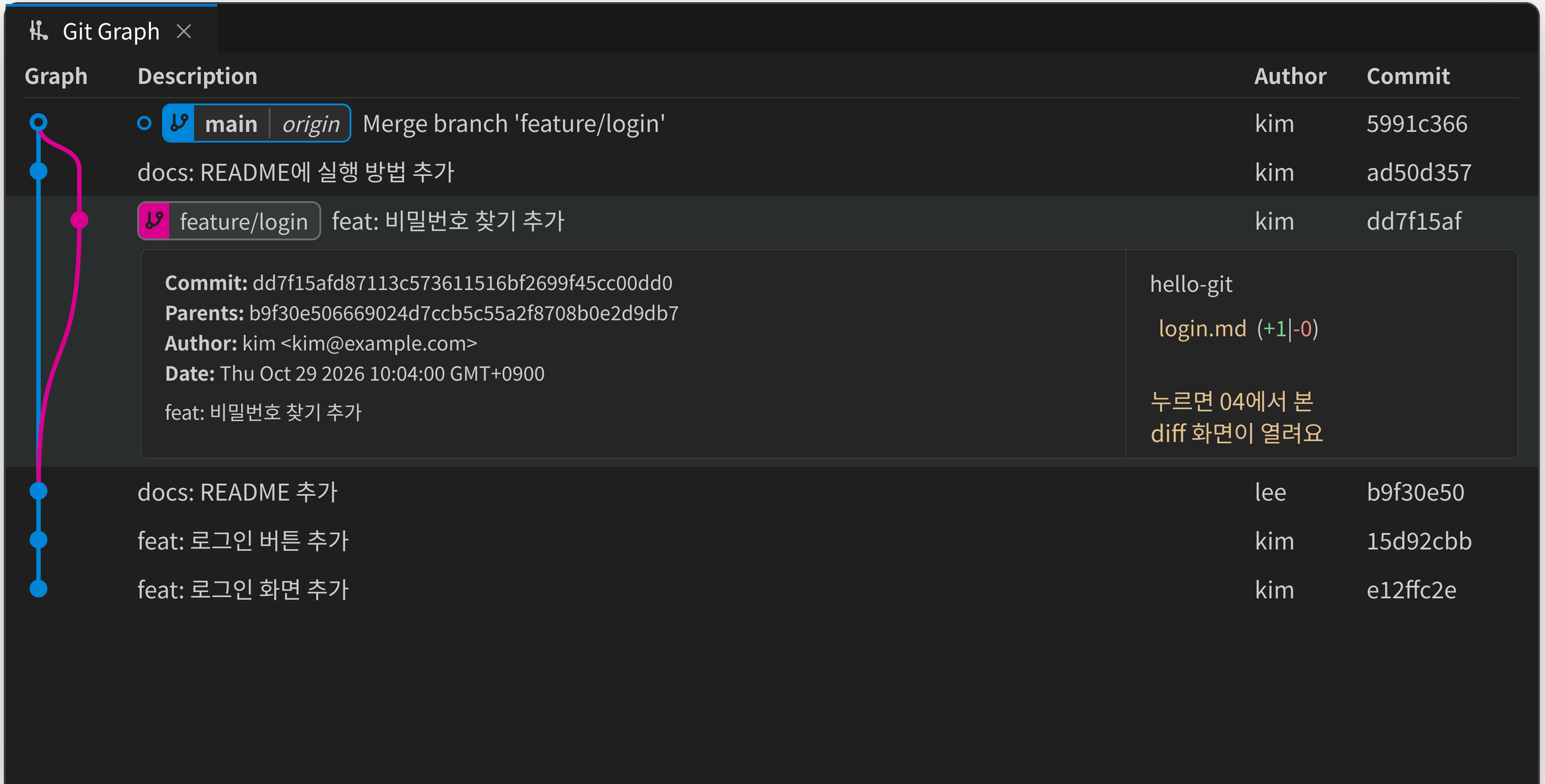
Graph	Description	Author	Commit
	main | origin Merge branch 'feature/login'	kim	5991c366
	docs: README에 실행 방법 추가	kim	ad50d357
	feature/login feat: 비밀번호 찾기 추가	kim	dd7f15af
	docs: README 추가	lee	b9f30e50
	feat: 로그인 버튼 추가	kim	15d92cbb
	feat: 로그인 화면 추가	kim	e12ffc2e

터미널

```
$ git push origin main
To https://github.com/kim/hello-git.git
   b9f30e5..5991c36  main -> main
```

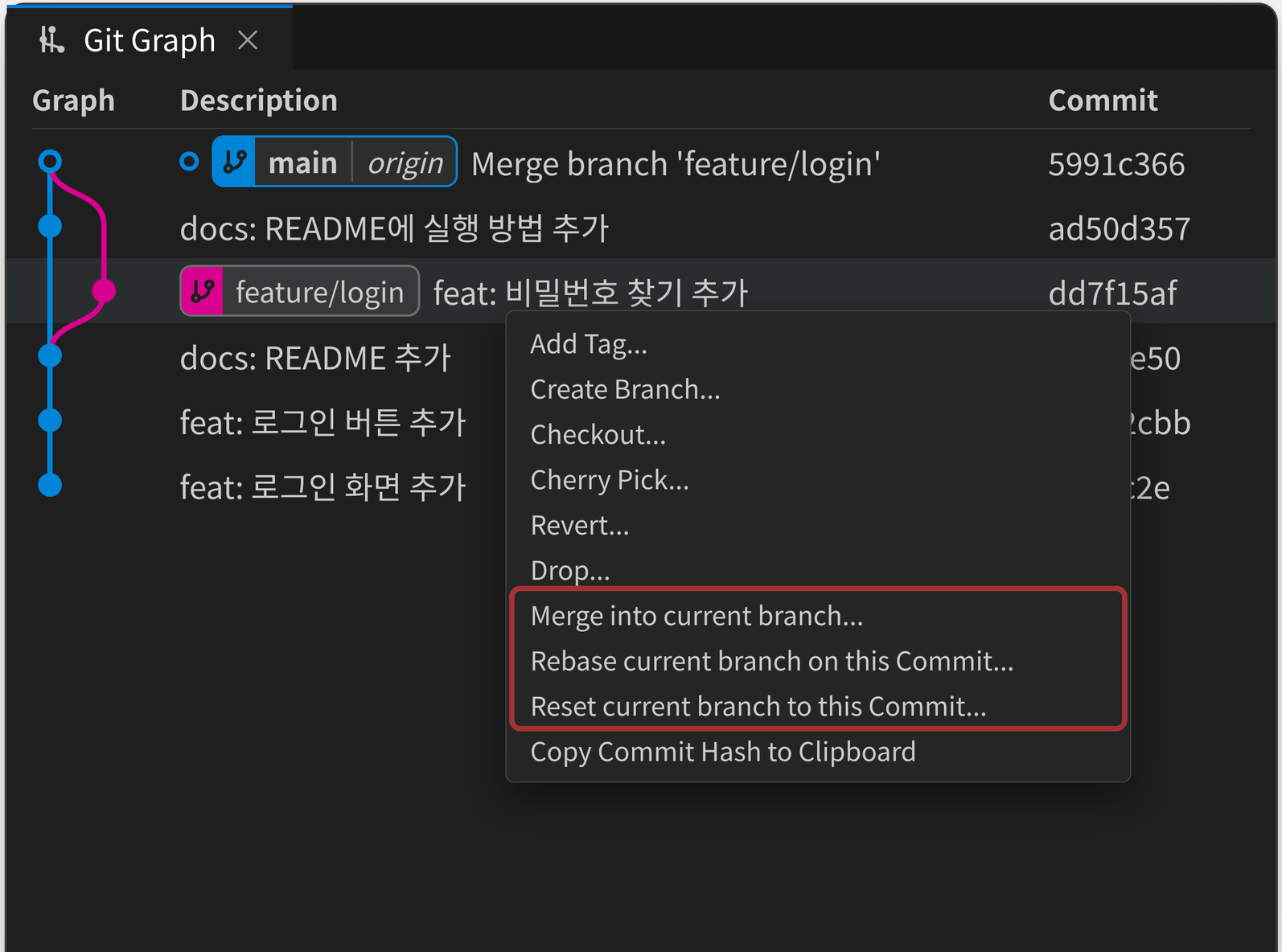
명령 치고 확인하기

커밋 줄을 누르면 바뀐 파일이 보여요



쓸 때 주의

checkout, merge, reset은 메뉴 말고 터미널에 쳐요



메뉴 한 번에 merge, reset이 실행돼요

터미널에 남아야 오류가 났을 때 읽고 고칠 수 있어요.

명령은 터미널

확인은 Git Graph

Fetch 버튼 대신 `git fetch origin`

막힐 때

Git Graph가 안 열리거나 그림이 그대로일 때

증상	왜	이렇게
저장소가 없다고 나와요 `No Git repositories were found...`	저장소가 아닌 폴더를 열었어요	**File > Open Folder로 hello-git 폴더 열기**
팀원이 올린 커밋이 안 보여요	내 컴퓨터에 받아 둔 기록만 그려요	**`git fetch origin`**
명령을 쳤는데 그림이 그대로예요	보통 저절로 바뀌어요	**오른쪽 위 새로고침 ↻**

실제 상황 1

그림만 보고 다음에 칠 명령을 골라요



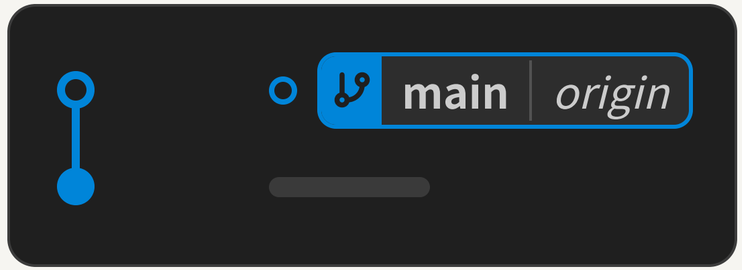
내가 커밋하고 아직 안 올렸어요

```
git push origin main
```



팀원 커밋을 fetch로 받아 왔어요

```
git pull origin main
```



둘이 붙어 있어요

```
할 일 없음
```

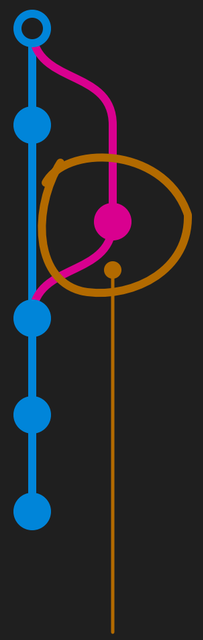
위에 있는 쪽이 앞서요. main이 위면 push, origin/main이 위면 pull이에요.

origin/main이 안 움직였는데 팀원이 올렸다고 하면, 먼저 git fetch origin을 쳐서 그림을 새로 받아요.

실제 상황 2

다 쓴 브랜치를 지우면 이름표만 사라져요

Git Graph

Graph	Description	Author	Commit
	main origin Merge branch 'feature/login'	kim	5991c366
	docs: README에 실행 방법 추가	kim	ad50d357
	feat: 비밀번호 찾기 추가	kim	dd7f15af
	docs: README 추가	lee	b9f30e50
	feat: 로그인 버튼 추가	kim	15d92cbb
	feat: 로그인 화면 추가	kim	e12ffc2e

점과 선은 그대로 남아요.
main의 기록에 이미 들어가 있어서예요.

터미널

```
$ git branch -d feature/login
Deleted branch feature/login (was dd7f15a).
```

정리

명령을 칠 때마다 Git Graph에서 한 곳씩 봐요

확인할 것	Git Graph에서 볼 곳	관련 명령
커밋했는지	맨 위 회색 줄이 사라지고 새 점	`git commit`
GitHub에 올렸는지	main | origin으로 붙었는지	`git push origin main`
팀원이 올린 게 있는지	origin/main이 main보다 위	`git fetch origin`
지금 어느 브랜치인지	굵은 글씨, 색 테두리 이름표	`git checkout 브랜치`
합쳐졌는지	두 선이 만나는 점	`git merge 브랜치`
무엇이 바뀌었는지	커밋 줄을 눌러 파일 목록	`git show`

다음 06에서는 kim, lee, park이 팀 레포에서 같이 작업해요. 서로 올린 커밋을 Git Graph로 확인하면서 충돌을 풀어요.